Servicenow

1) Filter “Assigned To” based on “Assignment Group”

Business Value:

In any organization, there are multiple teams:

- IT Support
- HR
- Finance

Without filtering:

👉 An Incident might be assigned to the wrong person ❌

1) Filter “Assigned To” based on “Assignment Group”

Benefits:

Reduces human errors

Faster ticket resolution

Improves SLA performance

Without this:

- Tickets go to wrong teams
- Delays in resolution
- Poor user satisfaction

1) Filter Assigned To based on Assignment Group

Concept:

In the **Incident form**:

•When the user selects an **Assignment Group**

•The **Assigned To** field should show only users who belong to that group

Implementation Options:

•**Client Script (onChange)**

•**Reference Qualifier (best practice)**

Example (Reference Qualifier):

```
javascript: 'sys_user_grmember.group=' + current.assignment_group
```

1) Filter “Assigned To” based on “Assignment Group”

Real Scenario:

User creates an Incident

Selects **Assignment Group = IT Support**

The system filters **Assigned To** → shows only IT users

1) Filter “Assigned To” based on “Assignment Group”

Section 1: Filtering Logic

Q1:

In a Management System:

There is a field **Department**

And a field **Employee**

👉 Make the Employee field show only users from the selected Department

Which method will you use?

Write the script

2) Attachments Handling (GlideSysAttachment Copy)

Business Value:

Attachments usually include:

- Screenshots
- Error logs
- Important documents

2) Attachments Handling (GlideSysAttachment Copy)

Why copy attachments?

•Problem team needs full context

•Avoid asking the user to upload again

Why “copy only once”?

If duplicated:

Database size increases unnecessarily

Same files repeated multiple times

Performance issues

2) Attachments Handling (GlideSysAttachment Copy)

Business Impact:

Saves time

Improves collaboration

Optimizes system performance

2) Attachments Handling (GlideSysAttachment Copy)

Concept:

When creating a **Problem** from an **Incident**:

- Copy attachments from Incident → Problem
- Ensure it happens **only once (no duplicates)**

Implementation:

Using:

GlideSysAttachment().copy()

Example:

```
var att = new GlideSysAttachment();
att.copy('incident', current.sys_id, 'problem', problemSysId);
```

Prevent Duplication:

•Add a custom field like: `u_attachments_copied = true`

•Or check if attachments already exist before copying

2) Attachments Handling (GlideSysAttachment Copy)

Real Scenario:

An Incident contains:

Error screenshot

Log file

When creating a **Problem**:

👉 The same files should be available

2) Attachments Handling (GlideSysAttachment Copy)

Section 2: Attachments Handling

Q2:

In a Task Management System:

When creating a **Project from Task**

👉 Copy attachments from Task → Project

BUT only once

What class will you use?

How will you prevent duplication?

3) Background Script (Data Creation / Update)

Business Value:

Instead of manually creating:

👉 100+ records

You automate it in seconds

Use Cases:

- Training environments
- Testing
- Bulk data creation

3) Background Script (Data Creation / Update)

Concept:

Generate sample data for testing or training

Implementation:

Using **Background Script**

Example:

```
for (var i = 1; i <= 10; i++) {
    var inc = new GlideRecord('incident');
    inc.initialize();
    inc.short_description = 'Training Incident ' + i;
    inc.insert();
}
```

3) Background Script (Data Creation / Update)

Real Scenario:

A company launches a new system
👉 Needs sample data for testing

3) Background Script (Data Creation / Update)

Section 3: Data Creation

Q3:

Create 10 sample records in table:

u_project

Each record should have:

Name: "Test Project X"

👉 Write a Background Script

4) Conditional Update (Update Only)

Business Value:

Update records **safely and intelligently**

Concept:

Update specific records based on conditions

Requirement:

•Update all **active Incidents**

•Where **Assignment Group is empty**

Example:

```
var gr = new GlideRecord('incident');
gr.addQuery('active', true);
gr.addNullQuery('assignment_group');
gr.query();

while (gr.next()) {
    gr.assignment_group = 'GROUP_SYS_ID';
    gr.update();
}
```

4) Conditional Update (Update Only)

Benefits:

- Clean and organized data
- No lost or unassigned tickets

Important:
Without conditions:
❌ You might accidentally update ALL records

4) Conditional Update (Update Only)

Scenario:

Many Incidents have no Assignment Group

👉 Assign them automatically

4) Conditional Update (Update Only)

Section 4: Conditional Update

Q4:

Update all active Tasks where:

assigned_to is empty

👉 Write the query and script

5) Email Notification (CC / BCC)

Business Value:

Ensure all stakeholders are informed

Concept:
Send email notifications with:
•To
•CC
•BCC
Implementation:
Inside **Notification** → **Advanced Script**
Example:

```
email.addAddress("cc", "manager@example.com");
email.addAddress("bcc", "audit@example.com");
```

5) Email Notification (CC / BCC)

CC → visible to all recipients
BCC → hidden recipients

Benefits:

Transparency

Better tracking

Compliance (very important in enterprises)

5) Email Notification (CC / BCC)

Scenario:

Critical Incident

Need to notify:

- Manager (CC)
- Audit Team (BCC)

5) Email Notification (CC / BCC)

Section 5: Notifications

Q5:

Send an email when a Project is created:

To: Owner

CC: Manager

BCC: Audit Team

👉 How do you implement CC and BCC in script?

6) UI Action (Create Problem Button)

Business Value:

Replace multiple manual steps with one click

UI Action Script – Create Problem from Incident

Concept:

Create a button on the Incident form:

•When clicked → creates a **Problem**

•Links it back to the Incident

Example:

```
var prob = new GlideRecord('problem');
prob.initialize();
prob.short_description = current.short_description;

var problemSysId = prob.insert();

current.problem_id = problemSysId;
current.update();
```

6) UI Action (Create Problem Button)

What happens:

Problem is created

Linked to Incident

(Optionally) attachments copied

Benefits:

Faster workflow

Reduced errors

Automation

6) UI Action (Create Problem Button)

Scenario:

Agent notices repeated Incidents

👉 Clicks "**Create Problem**"

6) UI Action (Create Problem Button)

Section 6: UI Action

Q6:

Create a button on Task:

👉 "Create Project"

Requirements:

•Create new Project

•Copy short description

•Link Project back to Task

👉 Write the script

Summary (Business Perspective)

Feature	Business Value
Filter Assigned To	Prevents errors & improves speed
Attachments Copy	Preserves data & saves time
Background Script	Saves effort & time
Conditional Update	Ensures data quality
Email CC/BCC	Improves communication
UI Action	Enables automation

All these features support 3 main business goals:

1. **Accuracy**
2. **Speed**
3. **Automation**

Practice

Main Scenario: Handling a Customer Complaint

Step 1: Create Ticket (like Incident)

A customer submits a complaint:

“Order arrived cold”

👉 System creates a **Ticket**

Fields:

•Customer Name

•Order ID

•Issue Type

•Assignment Group

•Assigned To

Step 2: Filter Assigned To based on Assignment Group

Logic:

When selecting:

Assignment Group = **Delivery Team**

👉 The **Assigned To** field shows only delivery staf

Business Value:

•Prevent assigning complaint to wrong team (e.g., Kitchen)

•Faster handling

Step 3: Attachments Handling

Customer uploads:

Photo of cold food

👉 Attachment is saved with the Ticket

Later:

Manager decides to escalate → create **Issue (Problem)**

👉 System:

•Copies attachments from Ticket → Issue

•Ensures it's copied **only once**

Business Value:
•No need to re-upload files
•All teams have full context

Step 4: UI Action - "Create Issue"

A button appears on Ticket:

👉 **"Create Issue"**

On Click:

Create new Issue record

Copy:

- Description
- Attachments

Link Issue back to Ticket

Business Value:

•One-click escalation
•Faster workflow
•Less manual work

Step 5: Email Notification (CC / BCC)

When Issue is created:

System sends Email:

To: Operations Manager

CC: Branch Manager

BCC: Audit Team

Business Value:

- Everyone is informed
- Management visibility
- Compliance tracking

Step 6: Background Script – Data Creation

For training:

👉 System Admin creates 10 sample Tickets

Example:

"Late delivery"

"Wrong order"

"Cold food"

Business Value:
Train new employees
Test system easy

Step 7: Background Script – Conditional Update

Problem:

Some tickets are created without Assignment Group

👉 Admin runs script:

Find active tickets with empty Assignment Group

Assign them to **Customer Service**

Business Value:

•No lost complaints

•Better organization

Questions

1. Which of the following is a server-side script that executes whenever a record is inserted, updated, deleted, or queried?

A. Client Script

B. UI Policy

C. Business Rule

D. UI Action (Client-side)

Answer:C

Business Rules are the primary server-side logic triggers for database operations.

2. A developer needs to execute a script once to update 500 records on the Incident table. Which tool is most appropriate for this "one-off" task?

A. Scheduled Job

B. Background Script

C. Fix Script

D. Business Rule

Answer:B

Background Scripts allow developers to run arbitrary JavaScript on the server for quick, one-time data fixes.

3. In a UI Action, what must be checked to allow the script to execute both client-side and server-side code?

A. "Client" checkbox

B. "Form button" checkbox

C. "Onclick" field must be empty

D. "Server-side" checkbox

Answer:A

Checking the Client box allows the onclick field to run client-side code, while the main script block handles server-side logic.

4. Which type of Script Include is typically used with GlideAjax to be called from a Client Script?

A. On Demand / Classless

B. Extends an existing class (Client Callable)

C. Defined as a global variable

D. Server-side only

Answer:B

Client Callable Script Includes must extend AbstractAjaxProcessor to work with GlideAjax.

5. Which Business Rule type is best for calculating values that do not need to be displayed to the user immediately after saving a record?

A. Before

B. After

C. Display

D. Async

Answer:D

Async Business Rules run in the background after the user's transaction, improving performance for non-immediate tasks.

6. Which object is used in a Script Include to refer to the current class instance?

A. current

B. this

C. self

D. parent

Answer:B

In ServiceNow JavaScript classes (Script Includes), this is used to refer to variables or methods within the same class.

7. When creating a Client Callable Script Include, which class must it extend?

A. GlideRecord

B. GlideSystem

C. AbstractAjaxProcessor

D. GlideForm

Answer:C

To handle AJAX requests from the client, the Script Include must extend the AbstractAjaxProcessor class.

8. Which of the following can a UI Action NOT be displayed as?

A. Form Button

B. Context Menu

C. List Link

D. Field Decoration

Answer:D

UI Actions can be buttons, links, or context menus; Field Decorations are typically configured via Dictionary attributes or Client Scripts.

Thank you